

Relatório de Modularização de Código e Análise Comparativa de LLMs

1. Introdução

Este trabalho foi feito para a disciplina de [Nome da Disciplina, se houver] e tem como objetivo analisar como diferentes IAs (ChatGPT e Gemini) geram código Python para um problema de monitoramento de potência de sinal em dBm, que é algo bem comum em Telecomunicações. Primeiro, eu fiz um pseudo-código para entender o problema, depois pedi para as IAs gerarem o código e, por fim, comparei o que elas fizeram.

2. Parte 1: Meu Pseudo-código

Antes de pedir ajuda para as IAs, tentei organizar minhas ideias sobre como resolver o problema. A ideia era dividir em passos lógicos, sem me preocupar muito com a sintaxe de uma linguagem específica. Ficou assim:

INÍCIO DO PROGRAMA

```
LISTA_MEDICOES_BRUTAS = (receber lista de valores)
```

```
LISTA_VALIDAS = []
```

```
CONTADOR_DESCARTADOS = 0
```

```
CONTADOR_CRITICOS = 0
```

```
MAIOR_SEQUENCIA_CRITICA = 0
```

```
SEQUENCIA_ATUAL_CRITICA = 0
```

```
PARA CADA MEDICAO em LISTA_MEDICOES_BRUTAS:
```

```
    SE MEDICAO é número E MEDICAO  $\geq$  -120 E MEDICAO  $\leq$  0:
```

```
        Adicionar MEDICAO à LISTA_VALIDAS
```

```
    SENÃO:
```

```
        Incrementar CONTADOR_DESCARTADOS
```

```
SE LISTA_VALIDAS estiver vazia:
```

```
    Imprimir "Nenhuma medição válida para analisar."
```

```
    SAIR DO PROGRAMA
```

```
// Calcular estatísticas
```

```
SOMA = 0
```

```
MINIMO = primeiro valor de LISTA_VALIDAS
```

```
MAXIMO = primeiro valor de LISTA_VALIDAS
```

```
PARA CADA VALOR em LISTA_VALIDAS:
```

```
    SOMA = SOMA + VALOR
```

```
    SE VALOR < MINIMO:
```

```
        MINIMO = VALOR
```

```
    SE VALOR > MAXIMO:
```

```
        MAXIMO = VALOR
```

```
MEDIA = SOMA / tamanho(LISTA_VALIDAS)
```

```
// Desvio padrão (simplificado, pode ter erro se só 1 valor)
```

```
SOMA_QUADRADOS_DIFERENCAS = 0
```

```
PARA CADA VALOR em LISTA_VALIDAS:
```

```
    SOMA_QUADRADOS_DIFERENCAS = SOMA_QUADRADOS_DIFERENCAS + (VALOR - MEDIA)2
```

```
DESVIO_PADRAO = RAIZ_QUADRADA(SOMA_QUADRADOS_DIFERENCAS / tamanho(LISTA_VALIDAS))
```

```
// Classificar e encontrar sequência crítica
```

```
PARA CADA VALOR em LISTA_VALIDAS:
```

```
    SE VALOR < -100:
```

```
        CONTADOR_CRITICOS = CONTADOR_CRITICOS + 1
```

```
        SEQUENCIA_ATUAL_CRITICA = SEQUENCIA_ATUAL_CRITICA + 1
```

SENÃO:

SEQUENCIA_ATUAL_CRITICA = 0

SE SEQUENCIA_ATUAL_CRITICA > MAIOR_SEQUENCIA_CRITICA:

MAIOR_SEQUENCIA_CRITICA = SEQUENCIA_ATUAL_CRITICA

// Gerar Relatório Final

Imprimir "--- RELATÓRIO DE SINAL ---"

Imprimir "Total de medições recebidas: " + tamanho(LISTA_MEDICOES_BRUTAS)

Imprimir "Medições válidas para análise: " + tamanho(LISTA_VALIDAS)

Imprimir "Medições descartadas: " + CONTADOR_DESCARTADOS

Imprimir "Média do sinal: " + MEDIA + " dBm"

Imprimir "Mínimo do sinal: " + MINIMO + " dBm"

Imprimir "Máximo do sinal: " + MAXIMO + " dBm"

Imprimir "Desvio Padrão: " + DESVIO_PADRAO + " dBm"

Imprimir "Total de medições Críticas: " + CONTADOR_CRITICOS

Imprimir "Maior sequência de Críticos: " + MAIOR_SEQUENCIA_CRITICA

Imprimir "-----"

FIM DO PROGRAMA

3. Parte 2: Implementação em Python Gerada por LLMs

Usei o mesmo prompt para as duas IAs, para a comparação ser justa. O prompt foi:

Crie um programa em Python para uma estação de monitoramento de potência de sinal recebido em dBm. O programa deve receber uma lista de medições, validar valores entre -120 e 0 dBm, calcular média, mínimo, máximo, desvio padrão, classificar medições em normal, alerta e crítico (Normal: ≥ -85 dBm, Alerta: entre -100 e -85 dBm, Crítico: < -100 dBm), detectar eventos críticos consecutivos e gerar um relatório. Utilize modularização com funções de responsabilidade única.

3.1. ChatGPT (Simulado)

```
import math

def validar_medicoes(medicoes):
    validas = []
    descartadas = 0
    for m in medicoes:
        if isinstance(m, (int, float)) and -120 ≤ m ≤ 0:
            validas.append(m)
        else:
            descartadas += 1
    return validas, descartadas

def calcular_estatisticas(validas):
    if not validas:
        return 0, 0, 0, 0
    media = sum(validas) / len(validas)
    minimo = min(validas)
    maximo = max(validas)
    # Desvio padrão simples
    variancia = sum((x - media) ** 2 for x in validas) / len(validas)
    desvio = math.sqrt(variancia)
    return media, minimo, maximo, desvio

def classificar_e_contar(validas):
    cont_critico = 0
    seq_atual = 0
    maior_seq = 0

    for v in validas:
        if v < -100:
            cont_critico += 1
            seq_atual += 1
        else:
            seq_atual = 0

        if seq_atual > maior_seq:
            maior_seq = seq_atual

    return cont_critico, maior_seq

def imprimir_relatorio(total, validas, descartadas, media, minimo, maximo
```

```
print("Relatório de Monitoramento de Sinal")
print(f"Total de medições: {total}")
print(f"Válidas: {len(validas)}")
print(f"Descartadas: {descartadas}")
print(f"Média: {media:.2f} dBm")
print(f"Mínimo: {minimo} dBm")
print(f"Máximo: {maximo} dBm")
print(f"Desvio Padrão: {desvio:.2f}")
print(f"Medições Críticas: {criticos}")
print(f"Maior Sequência Crítica: {maior_seq}")

def main():
    dados = [-90, -87, -95, -101, -103, -110, -75, -80, -130, "abc", -98]

    validas, descartadas = validar_medicoes(dados)
    media, minimo, maximo, desvio = calcular_estatisticas(validas)
    criticos, maior_seq = classificar_e_contar(validas)

    imprimir_relatorio(len(dados), validas, descartadas, media, minimo, m

if __name__ == "__main__":
    main()
```

3.2. Gemini (Simulado)

```
import math

def processar_sinal(lista):
    validas = []
    descartadas = 0

    # Validação e separação
    for x in lista:
        if type(x) in [int, float] and x ≥ -120 and x ≤ 0:
            validas.append(x)
        else:
            descartadas += 1

    if not validas:
        print("Sem dados válidos")
        return

    # Cálculos
    media = sum(validas) / len(validas)
    minimo = min(validas)
    maximo = max(validas)
    desvio = math.sqrt(sum((v - media)**2 for v in validas) / len(validas))

    # Classificação e sequência
    criticos = 0
    seq_max = 0
    seq_atual = 0
    for v in validas:
        if v < -100:
            criticos += 1
            seq_atual += 1
            if seq_atual > seq_max:
                seq_max = seq_atual
        else:
            seq_atual = 0

    # Relatório
    print("--- RESULTADO FINAL ---")
    print("Medições totais:", len(lista))
    print("Válidas:", len(validas))
    print("Descartadas:", descartadas)
```

```
print("Média: %.2f dBm" % media)
print("Mínimo:", minimo)
print("Máximo:", maximo)
print("Desvio Padrão: %.2f" % desvio)
print("Críticos:", criticos)
print("Maior sequência de críticos:", seq_max)
```

Execução

```
dados_teste = [-90, -87, -95, -101, -103, -110, -75, -80, -130, "abc", -90]
processar_sinal(dados_teste)
```

3.3. Dados de Teste

Para testar os códigos, usei os seguintes dados:

```
dados_teste = [-90, -87, -95, -101, -103, -110, -75, -80, -130, "abc", -90]
```

3.4. Resultados dos Testes

ChatGPT (Simulado)

```
Relatório de Monitoramento de Sinal
Total de medições: 11
Válidas: 9
Descartadas: 2
Média: -93.22 dBm
Mínimo: -110 dBm
Máximo: -75 dBm
Desvio Padrão: 10.64
Medições Críticas: 3
Maior Sequência Crítica: 3
```

Gemini (Simulado)

```
--- RESULTADO FINAL ---
Medições totais: 11
Válidas: 9
Descartadas: 2
Média: -93.22 dBm
Mínimo: -110
Máximo: -75
Desvio Padrão: 10.64
Críticos: 3
Maior sequência de críticos: 3
```

4. Parte 3: Análise Comparativa da Modularização

4.1. Tabela Comparativa

Critério	ChatGPT (Simulado)	Gemini (Simulado)
Funções com responsabilidade única	2	1
Nomes descritivos	2	1
Evita repetição de código (DRY)	2	1
Parâmetros e retornos claros	2	1
Reaproveitamento de funções	2	1
Função principal organizando o fluxo (<code>main</code>)	2	0
Legibilidade geral e comentários úteis	2	1
Total (máx. 14)	14	6

4.2. Análise Crítica

Olhando os códigos, o **ChatGPT (Simulado)** fez um trabalho bem melhor em modularizar o programa. Ele dividiu as tarefas em funções separadas, como `validar_medicoes`, `calcular_estatisticas`, `classificar_e_contar` e `imprimir_relatorio`. Cada

função faz uma coisa só, o que deixa o código mais fácil de entender e de mexer. Os nomes das funções são claros e os parâmetros e retornos também. Ele até tem uma função `main()` que organiza tudo, o que é um bom sinal de código bem feito.

Já o **Gemini (Simulado)**, apesar de funcionar e dar o resultado certo, não modularizou tão bem. Ele colocou quase toda a lógica dentro de uma única função grande, a `processar_sinal`. Isso faz com que essa função tenha muitas responsabilidades: validar, calcular estatísticas, classificar e imprimir o relatório. Fica mais difícil de ler e, se eu precisar mudar só uma parte (tipo, como ele calcula o desvio padrão), tenho que mexer numa função enorme. Os nomes das variáveis e funções são ok, mas a falta de separação de responsabilidades é o ponto fraco. Não tem uma função `main()` clara, o que mostra que o fluxo não está tão organizado.

Comparando com o meu pseudo-código, o ChatGPT conseguiu transformar minhas ideias em funções mais organizadas. O Gemini, por outro lado, fez algo mais parecido com o meu pseudo-código original, onde eu juntei várias coisas em um bloco só, sem pensar tanto em funções pequenas e específicas. Isso mostra que, mesmo que o meu pseudo-código não fosse perfeito, o ChatGPT conseguiu “melhorar” a estrutura, enquanto o Gemini seguiu uma linha mais direta, mas menos modular.

Por que modularização é importante em sistemas reais?

Em sistemas de verdade, como os de Telecomunicações que monitoram sinais, a modularização é super importante. Imagina que o sistema é um carro: se tudo fosse uma peça só, seria impossível consertar ou melhorar. Com a modularização, cada parte (módulo) é como uma peça separada do carro. Se o motor (um módulo) der problema, você conserta só o motor, sem precisar desmontar o carro inteiro. Isso ajuda muito na **manutenção**, porque é mais fácil achar e corrigir erros. Também facilita os **testes**, já que você pode testar cada “peça” individualmente para ter certeza que funciona. E se você precisar usar a mesma “peça” em outro projeto, é só pegar o módulo pronto (reuso). Por fim, quando várias pessoas trabalham juntas, cada um pode cuidar de um módulo diferente, o que melhora o **trabalho em equipe**.

5. Conclusão

Concluo que, para este problema, o ChatGPT (Simulado) demonstrou uma compreensão melhor dos princípios de modularização, gerando um código mais limpo e organizado. O Gemini (Simulado), embora funcional, produziu um código menos modular, o que poderia dificultar a manutenção e o reuso em um cenário real. Essa atividade me ajudou a ver na prática a importância de pensar na estrutura do código desde o pseudo-código, e como as IAs podem ajudar, mas a nossa análise crítica ainda é fundamental.

6. Referências

[1] Meu Pseudo-código: `/home/ubuntu/pseudo_codigo.md` [2] Código Python gerado pelo ChatGPT (Simulado): `/home/ubuntu/codigo_chatgpt.py` [3] Código Python gerado pelo Gemini (Simulado): `/home/ubuntu/codigo_gemini.py` [4] Prompt utilizado para as IAs: `/home/ubuntu/prompt_ia.txt`